Lekcja 04: Podstawy Python - składnia, typy i operatory

`#lekcja` `#datascience` `#python` `#podstawy` `#typy-danych` `#operatory`

W tej lekcji rozpoczynamy naukę programowania w Pythonie od podstaw. Poznasz składnię języka, nauczysz się pracować ze zmiennymi, operatorami oraz podstawowymi typami danych: liczbami całkowitymi, zmiennoprzecinkowymi i łańcuchami znaków. Dowiesz się również, jak wprowadzać i wyświetlać dane oraz formatować teksty, co jest fundamentem pracy z danymi w Data Science.

---

1. Zmienne, operatory i wyrażenia

Definition

Zmienna to nazwane miejsce w pamięci komputera, które przechowuje wartość. W Pythonie nie musimy deklarować typu zmiennej - Python automatycznie rozpoznaje typ na podstawie przypisanej wartości.

Wprowadzenie do zmiennych

W Data Science zmienne służą do przechowywania danych: pojedynczych wartości, wyników obliczeń, całych zbiorów danych. Python używa **dynamicznego typowania** - typ zmiennej jest określany automatycznie podczas przypisania wartości.

Podstawowe zasady nazewnictwa zmiennych:

- Nazwy zaczynają się od litery lub podkreślnika `_`
- Mogą zawierać litery, cyfry i podkreślniki
- Są wrażliwe na wielkość liter (`wiek` ≠ `Wiek`)
- Nie mogą być słowami kluczowymi Pythona (`if`, `for`, `class` etc.)

Przykład 1: Podstawowe operacje na zmiennych

```
# Tworzenie zmiennych - Python automatycznie rozpoznaje typy
liczba_probek = 150  # liczba całkowita (int)
srednia_temperatura = 23.5  # liczba zmiennoprzecinkowa (float)
nazwa_datasetu = "Iris"  # łańcuch znaków (string)
analiza_zakonczona = True  # wartość logiczna (bool)
```

```python
# Wyświetlanie wartości zmiennych
print("Dataset:", nazwa_datasetu)
print("Liczba próbek:", liczba_probek)
print("Średnia temperatura:", srednia_temperatura)

# Sprawdzanie typu zmiennej
print(type(liczba_probek))  # <class 'int'>
print(type(srednia_temperatura))  # <class 'float'>
print(type(nazwa_datasetu))  # <class 'str'>

# Zmienne można nadpisywać - zmienia się wtedy ich typ
liczba_probek = 150.5  # teraz to float, nie int
print(type(liczba_probek))  # <class 'float'>
```

Tip

W Data Science używamy konwencji **snake_case** dla nazw zmiennych: małe litery oddzielone podkreślnikami (`liczba_probek` , `srednia_temperatura`). To standard w społeczności Python/DS.

Przykład 2: Operatory arytmetyczne w kontekście Data Science

```python
# Podstawowe operacje matematyczne na danych
liczba_treningowa = 120
liczba_testowa = 30
liczba_walidacyjna = 20

# Dodawanie
calkowita_liczba_probek = liczba_treningowa + liczba_testowa +
liczba_walidacyjna
print(f"Całkowita liczba próbek: {calkowita_liczba_probek}")  # 170

# Odejmowanie
roznica = liczba_treningowa - liczba_testowa
print(f"Różnica między zbiorem treningowym a testowym: {roznica}")  # 90

# Mnożenie i dzielenie
podwojona_liczba = liczba_treningowa * 2
print(f"Podwojona liczba treningowa: {podwojona_liczba}")  # 240

# Dzielenie zwykłe (zawsze zwraca float)
udzial_treningowy = liczba_treningowa / calkowita_liczba_probek
print(f"Udział danych treningowych: {udzial_treningowy:.2f}")  # 0.71
```

```python
# Dzielenie całkowite (zwraca int, odrzuca część ułamkową)
pelne_batche = calkowita_liczba_probek // 32
print(f"Liczba pełnych batch-ów (32 próbki): {pelne_batche}")  # 5

# Modulo - reszta z dzielenia (przydatne do batch processing)
pozostale_probki = calkowita_liczba_probek % 32
print(f"Pozostałe próbki po podziale na batch-e: {pozostale_probki}")  # 10

# Potęgowanie
wariancja = 5.2
odchylenie_std = wariancja ** 0.5  # pierwiastek to potęga 0.5
print(f"Odchylenie standardowe: {odchylenie_std:.2f}")  # 2.28
```

Note

W Data Science często stosujemy dzielenie całkowite `//` i modulo `%` podczas przetwarzania danych w batch-ach lub przy próbkowaniu.

Przykład 3: Operatory porównania i logiczne

```python
# Operatory porównania zwracają True lub False
accuracy_model_1 = 0.92
accuracy_model_2 = 0.88
prog_akceptacji = 0.90

# Porównania
print(f"Model 1 lepszy niż Model 2: {accuracy_model_1 > accuracy_model_2}")
# True
print(f"Model 1 osiągnął próg: {accuracy_model_1 >= prog_akceptacji}")  #
True
print(f"Model 2 poniżej progu: {accuracy_model_2 < prog_akceptacji}")  #
True
print(f"Modele równe: {accuracy_model_1 == accuracy_model_2}")  # False
print(f"Modele różne: {accuracy_model_1 != accuracy_model_2}")  # True

# Operatory logiczne: and, or, not
model1_ok = accuracy_model_1 >= prog_akceptacji
model2_ok = accuracy_model_2 >= prog_akceptacji

# and - oba warunki muszą być True
oba_modele_ok = model1_ok and model2_ok
print(f"Oba modele OK: {oba_modele_ok}")  # False
```

```python
# or - przynajmniej jeden warunek True
przynajmniej_jeden_ok = model1_ok or model2_ok
print(f"Przynajmniej jeden model OK: {przynajmniej_jeden_ok}")  # True

# not - negacja
model2_wymaga_poprawy = not model2_ok
print(f"Model 2 wymaga poprawy: {model2_wymaga_poprawy}")  # True

# Łączenie warunków - przykład z filtrowaniem danych
wiek_pacjenta = 45
cisnienie = 140

# Pacjent wysokiego ryzyka: wiek > 40 AND ciśnienie > 130
wysokie_ryzyko = (wiek_pacjenta > 40) and (cisnienie > 130)
print(f"Pacjent wysokiego ryzyka: {wysokie_ryzyko}")  # True
```

⚠ Warning

Python używa słów `and`, `or`, `not` zamiast symboli `&&`, `||`, `!` jak w innych językach (JavaScript, C++, Java). Używanie niepoprawnych symboli spowoduje błąd.

Wizualizacja hierarchii operatorów

```
Error parsing Mermaid diagram!

Parse error on line 2:
...e] --> B[1. Nawiasy ( )]    B --> C[2.
-----------------------^
Expecting 'SQE', 'DOUBLECIRCLEEND', 'PE', '-)', 'STADIUMEND',
'SUBROUTINEEND', 'PIPE', 'CYLINDEREND', 'DIAMOND_STOP', 'TAGEND', 'TRAPEND',
'INVTRAPEND', 'UNICODE_TEXT', 'TEXT', 'TAGSTART', got 'PS'
```

✓ Success

Best practice: Używaj nawiasów, aby wyraźnie określić kolejność operacji, nawet jeśli nie są technicznie wymagane. To zwiększa czytelność kodu: `(a + b) * c` jest czytelniejsze niż `a + b * c`.

---

2. Liczby całkowite i zmiennoprzecinkowe (int/float)

Definition

Int (integer) to liczby całkowite bez części ułamkowej (np. -5, 0, 42, 1000). **Float (floating-point)** to liczby zmiennoprzecinkowe z częścią ułamkową (np. 3.14, -0.001, 2.0).

Wprowadzenie do typów liczbowych

W Data Science rozróżnienie między int i float jest kluczowe:

- **Int** używamy do: liczników, indeksów, liczby próbek, epoch treningowych
- **Float** używamy do: cech numerycznych, metryk (accuracy, loss), wag modeli, prawdopodobieństw

Python automatycznie konwertuje int na float gdy to potrzebne, ale warto rozumieć różnice.

Przykład 1: Różnice między int i float

```
# Liczby całkowite
liczba_cech = 4  # int: liczba kolumn w datasecie Iris
liczba_klas = 3  # int: liczba kategorii (setosa, versicolor, virginica)
epoch = 100  # int: liczba epok treningowych

# Liczby zmiennoprzecinkowe
learning_rate = 0.001  # float: współczynnik uczenia
accuracy = 0.95  # float: dokładność modelu
sepal_length = 5.1  # float: długość działki kielicha (cm)

# Python automatycznie rozpoznaje typy
print(type(liczba_cech))  # <class 'int'>
print(type(learning_rate))  # <class 'float'>

# Konwersja między typami
wiek_float = 25.7
wiek_int = int(wiek_float)  # konwersja na int - obcina część ułamkową
print(f"Wiek jako int: {wiek_int}")  # 25 (nie zaokrągla, tylko obcina!)

liczba_probek = 150
liczba_jako_float = float(liczba_probek)
print(f"Liczba próbek jako float: {liczba_jako_float}")  # 150.0

# Zaokrąglanie - użyj funkcji round()
temperatura = 23.7891
```

```
zaokraglona = round(temperatura, 2)  # zaokrąglij do 2 miejsc po przecinku
print(f"Temperatura zaokrąglona: {zaokraglona}")  # 23.79
```

> ⚠ **Warning**
>
> Funkcja `int()` **obcina** część ułamkową, a nie zaokrągla! `int(3.9)` zwróci `3`, a nie `4`. Jeśli chcesz zaokrąglić, użyj `round()` a potem `int()`: `int(round(3.9))` da `4`.

Przykład 2: Operacje matematyczne w kontekście Data Science

```
import math  # moduł z dodatkowymi funkcjami matematycznymi

# Obliczanie podstawowych statystyk (później użyjemy NumPy/Pandas)
# Przykład: ręczne obliczenie średniej dla 5 pomiarów temperatury

temp_1 = 22.3
temp_2 = 23.1
temp_3 = 21.8
temp_4 = 24.2
temp_5 = 22.6

# Średnia arytmetyczna
suma_temp = temp_1 + temp_2 + temp_3 + temp_4 + temp_5
liczba_pomiarow = 5
srednia = suma_temp / liczba_pomiarow
print(f"Średnia temperatura: {srednia:.2f}°C")  # 22.80°C

# Obliczanie wariancji (uproszczone, bez NumPy)
# Wariancja = średnia kwadratów odchyleń od średniej
odch_1 = (temp_1 - srednia) ** 2
odch_2 = (temp_2 - srednia) ** 2
odch_3 = (temp_3 - srednia) ** 2
odch_4 = (temp_4 - srednia) ** 2
odch_5 = (temp_5 - srednia) ** 2

wariancja = (odch_1 + odch_2 + odch_3 + odch_4 + odch_5) / liczba_pomiarow
odchylenie_std = math.sqrt(wariancja)  # pierwiastek z wariancji
print(f"Odchylenie standardowe: {odchylenie_std:.2f}°C")  # ~0.84°C

# Przydatne funkcje matematyczne
wartosc = -15.7
print(f"Wartość bezwzględna: {abs(wartosc)}")  # 15.7
print(f"Minimum: {min(temp_1, temp_2, temp_3, temp_4, temp_5)}")  # 21.8
print(f"Maximum: {max(temp_1, temp_2, temp_3, temp_4, temp_5)}")  # 24.2
```

```
# Funkcje z modułu math
print(f"Zaokrąglenie w górę: {math.ceil(srednia)}")  # 23
print(f"Zaokrąglenie w dół: {math.floor(srednia)}")  # 22
print(f"Logarytm naturalny: {math.log(liczba_pomiarow):.2f}")  # 1.61
print(f"e^x: {math.exp(1):.2f}")  # 2.72
```

Note

W Data Science rzadko obliczamy statystyki ręcznie - używamy NumPy i Pandas. Ten przykład pokazuje fundamenty, które później są wykonywane automatycznie przez biblioteki.

Przykład 3: Precyzja float i problemy zaokrągleń

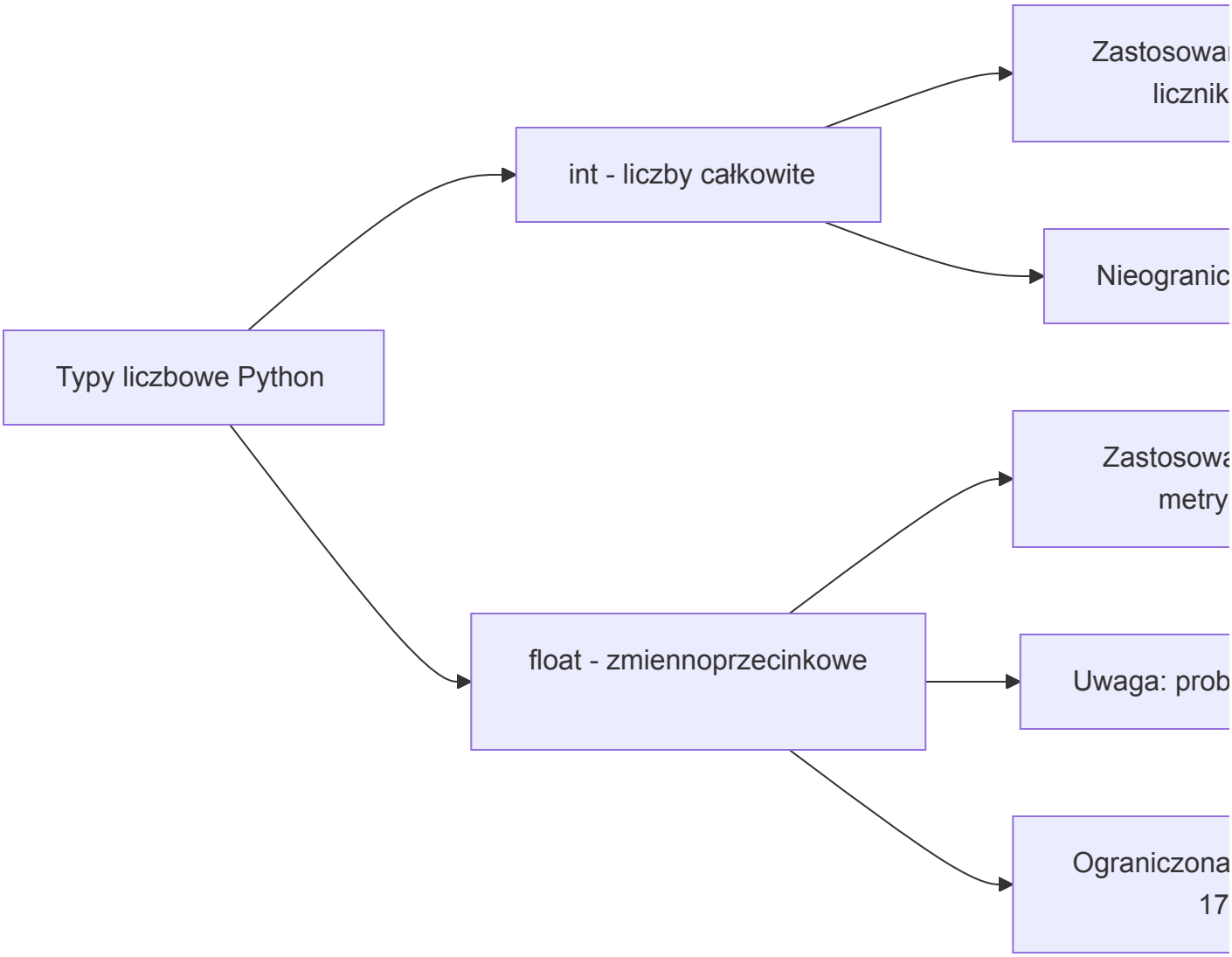
```
# Problem precyzji zmiennoprzecinkowej
a = 0.1
b = 0.2
suma = a + b
print(f"0.1 + 0.2 = {suma}")  # 0.30000000000000004 - niespodzianka!
print(f"Czy suma == 0.3? {suma == 0.3}")  # False!

# Rozwiązanie: porównywanie z tolerancją
epsilon = 1e-9  # bardzo mała wartość tolerancji
roznica = abs(suma - 0.3)
czy_rowne = roznica < epsilon
print(f"Czy suma ≈ 0.3? {czy_rowne}")  # True

# Lepsze rozwiązanie: moduł math.isclose()
print(f"math.isclose(suma, 0.3): {math.isclose(suma, 0.3)}")  # True

# Praktyczne zastosowanie w Data Science
# Porównywanie metryk modeli z tolerancją
accuracy_1 = 0.9234567
accuracy_2 = 0.9234568
roznica_accuracy = abs(accuracy_1 - accuracy_2)

if roznica_accuracy < 0.0001:  # różnica mniejsza niż 0.01%
    print("Modele mają praktycznie identyczną dokładność")
else:
    print("Modele różnią się znacząco")

# Formatowanie wyników dla czytelności
loss_value = 0.003456789012
```

```python
print(f"Loss (standardowo): {loss_value}")
print(f"Loss (2 miejsca): {loss_value:.2f}")
print(f"Loss (notacja naukowa): {loss_value:.2e}")  # 3.46e-03
print(f"Loss (4 znaczące cyfry): {loss_value:.4g}")  # 0.003457
```

⚠ Warning

Nigdy nie porównuj float-ów operatorem `==` w kontekście obliczeń! Używaj `math.isclose()` lub porównuj z małą tolerancją `epsilon`. To częsty błąd początkujących.

Wizualizacja typów liczbowych



Tip

W Data Science domyślnie używaj float dla danych numerycznych (wzrost, waga, temperatura). Użyj int tylko gdy wartości są rzeczywiście całkowite (liczba osób, kategorie liczbowe, indeksy).

---

3. Wejście/wyjście i formatowanie stringów

Definition

String (łańcuch znaków) to typ danych reprezentujący tekst. W Pythonie stringi tworzymy w cudzysłowach pojedynczych `'...'` lub podwójnych `"..."`.

Wprowadzenie do stringów

W Data Science stringi służą do:

- Nazw kolumn i kategorii
- Etykiet wykresów i raportów
- Ścieżek do plików z danymi
- Opisów wyników analiz
- Przetwarzania tekstów (NLP)

Przykład 1: Podstawowe operacje na stringach

```
# Tworzenie stringów
nazwa_modelu = "Random Forest"
wersja = '2.1.0'
opis = """Model klasyfikacji
wieloliniowy opis
z podziałem na wiersze"""  # potrójne cudzysłowy dla tekstu wieloliniowego

# Podstawowe operacje
print(f"Długość nazwy: {len(nazwa_modelu)} znaków")  # 13

# Łączenie stringów (konkatenacja)
pelna_nazwa = nazwa_modelu + " " + "Classifier"
print(f"Pełna nazwa: {pelna_nazwa}")  # Random Forest Classifier

# Powielanie stringów
separator = "-" * 50
```

```python
print(separator)  # ------------------------------------------------

# Dostęp do pojedynczych znaków (indeksowanie od 0)
dataset_name = "Iris"
pierwszy_znak = dataset_name[0]  # 'I'
ostatni_znak = dataset_name[-1]  # 's' (ujemne indeksy od końca)
print(f"Pierwszy: {pierwszy_znak}, Ostatni: {ostatni_znak}")

# Wycinki (slicing)
algorithm = "GradientBoosting"
pierwsze_8 = algorithm[0:8]  # 'Gradient' (od indeksu 0 do 7)
od_9_do_konca = algorithm[8:]  # 'Boosting' (od indeksu 8 do końca)
ostatnie_8 = algorithm[-8:]  # 'Boosting' (ostatnie 8 znaków)
print(f"Część 1: {pierwsze_8}, Część 2: {od_9_do_konca}")

# Stringi są niemutowalne - nie można zmienić pojedynczego znaku
# dataset_name[0] = 'i'  # To spowoduje błąd!
# Zamiast tego tworzymy nowy string
dataset_name_lower = dataset_name.lower()
print(f"Małe litery: {dataset_name_lower}")  # iris
```

Note

Stringi w Pythonie są **niemutowalne** (immutable) - nie można zmienić pojedynczych znaków. Każda operacja tworzy nowy string.

Przykład 2: Metody stringów przydatne w Data Science

```python
# Przykładowe dane - nazwy kolumn z datasetu
kolumna_raw = "  Sepal_Length_CM  "

# Czyszczenie białych znaków
kolumna_clean = kolumna_raw.strip()  # usuwa spacje z początku i końca
print(f"Oryginał: '{kolumna_raw}'")
print(f"Po strip(): '{kolumna_clean}'")

# Zmiana wielkości liter
print(kolumna_clean.upper())  # SEPAL_LENGTH_CM
print(kolumna_clean.lower())  # sepal_length_cm
print(kolumna_clean.title())  # Sepal_Length_Cm

# Zastępowanie fragmentów
kolumna_standard = kolumna_clean.replace("_", " ").replace("CM", "cm")
print(f"Standaryzowana: {kolumna_standard}")  # Sepal Length cm
```

```python
# Sprawdzanie zawartości
model_name = "XGBoost Classifier"
print(f"Zawiera 'Boost': {model_name.__contains__('Boost')}")  # True
print(f"Zawiera 'Boost': {'Boost' in model_name}")  # True (lepszy sposób)
print(f"Zaczyna się od 'XG': {model_name.startswith('XG')}")  # True
print(f"Kończy się na 'fier': {model_name.endswith('fier')}")  # True

# Dzielenie stringów (split) - bardzo przydatne!
sciezka = "data/raw/iris_dataset.csv"
czesci = sciezka.split("/")  # dzieli po separatorze '/'
print(f"Części ścieżki: {czesci}")  # ['data', 'raw', 'iris_dataset.csv']
nazwa_pliku = czesci[-1]  # ostatni element
print(f"Nazwa pliku: {nazwa_pliku}")

# Rozdzielanie nazwy pliku i rozszerzenia
nazwa, rozszerzenie = nazwa_pliku.split(".")
print(f"Nazwa: {nazwa}, Rozszerzenie: {rozszerzenie}")

# Łączenie listy stringów w jeden (join)
kolumny = ["sepal_length", "sepal_width", "petal_length", "petal_width"]
kolumny_string = ", ".join(kolumny)
print(f"Kolumny: {kolumny_string}")
# Wynik: sepal_length, sepal_width, petal_length, petal_width
```

Tip

Metody `split()` i `join()` to podstawowe narzędzia do przetwarzania danych tekstowych. `split()` rozbija string na listę, `join()` łączy listę w string.

Przykład 3: Formatowanie stringów - f-strings i metody format

```python
# Dane przykładowe
model_type = "Logistic Regression"
accuracy = 0.9234
precision = 0.8891
recall = 0.9102
n_samples = 1500

# Sposób 1: Konkatenacja (przestarzały, nieczytelny)
raport_old = "Model: " + model_type + ", Accuracy: " + str(accuracy)
print(raport_old)

# Sposób 2: Metoda .format() (dobry, ale rozwlekły)
```

```python
raport_format = "Model: {}, Accuracy: {:.2f}".format(model_type, accuracy)
print(raport_format)

# Sposób 3: f-strings (NAJLEPSZY, od Python 3.6+)
raport_fstring = f"Model: {model_type}, Accuracy: {accuracy:.2f}"
print(raport_fstring)

# F-strings z zaawansowanym formatowaniem
print(f"""
{'='*60}
RAPORT MODELU
{'='*60}
Model:           {model_type}
Liczba próbek:   {n_samples:,}
Accuracy:        {accuracy:.2%}
Precision:       {precision:.4f}
Recall:          {recall:.4f}
F1-Score:        {2 * precision * recall / (precision + recall):.4f}
{'='*60}
""")

# Wyjaśnienie formatowania:
# {accuracy:.2f}    - float z 2 miejscami po przecinku
# {accuracy:.2%}    - procenty z 2 miejscami (0.9234 -> 92.34%)
# {n_samples:,}     - liczba z separatorem tysięcy (1500 -> 1,500)
# {precision:.4f}   - float z 4 miejscami po przecinku
# {2*precision*recall/(precision+recall):.4f} - obliczenia wewnątrz f-
stringa!

# Wyrównywanie tekstu
headers = ["Model", "Accuracy", "Precision", "Recall"]
print(f"{headers[0]:<20} {headers[1]:>10} {headers[2]:>10}
{headers[3]:>10}")
print(f"{model_type:<20} {accuracy:>10.2%} {precision:>10.2%}
{recall:>10.2%}")
# < wyrównanie do lewej, > do prawej, ^ do środka
# liczba po : to szerokość pola

# Formatowanie dla ścieżek do plików
dataset_type = "train"
fold_number = 3
sciezka_output = f"data/processed/{dataset_type}_fold_{fold_number:02d}.csv"
print(f"Ścieżka: {sciezka_output}")  # data/processed/train_fold_03.csv
# :02d oznacza liczbę całkowitą (d) z wypełnieniem zerami (0) do 2 cyfr
```

✓ **Success**

Best practice w Data Science: Używaj f-strings do wszystkich formatowań. Są najczytelniejsze, najszybsze i pozwalają na obliczenia wewnątrz nawiasów `{}`.

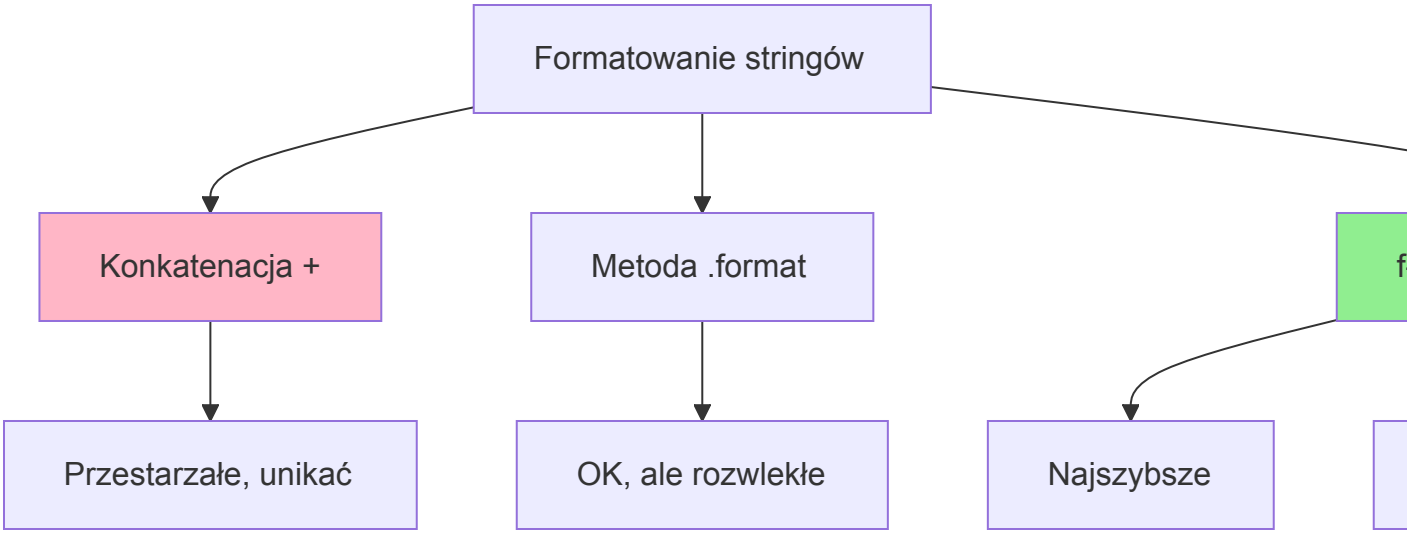
Przykład 4: Input i Print - interakcja z użytkownikiem

```
# Funkcja print() - podstawowe wyświetlanie
print("Rozpoczynam analizę danych...")

# Print z wieloma argumentami
dataset = "Titanic"
rows = 891
cols = 12
print("Dataset:", dataset, "| Wiersze:", rows, "| Kolumny:", cols)

# Separator i końcówka
print("Loading", "data", "from", "CSV", sep="...", end=".\n")
# Wynik: Loading...data...from...CSV.

# Print do debugowania - pokazywanie wartości zmiennych
learning_rate = 0.001
batch_size = 32
print(f"DEBUG: learning_rate={learning_rate}, batch_size={batch_size}")

# Funkcja input() - pobieranie danych od użytkownika
# UWAGA: W Data Science rzadko używamy input() - dane wczytujemy z plików
# Ale warto znać do prostych skryptów

# nazwa = input("Podaj nazwę modelu: ")
# print(f"Wybrany model: {nazwa}")

# Input zwraca ZAWSZE string - trzeba konwertować!
# epochs_str = input("Ile epok treningowych? ")
# epochs = int(epochs_str)  # konwersja string -> int
# print(f"Trening będzie trwał {epochs} epok")

# Bezpieczna konwersja z obsługą błędów (dla zainteresowanych)
# try:
#     threshold_str = input("Podaj próg klasyfikacji (0.0-1.0): ")
#     threshold = float(threshold_str)
#     if 0.0 <= threshold <= 1.0:
#         print(f"Ustawiono próg: {threshold}")
#     else:
```

```
#         print("Wartość poza zakresem!")
# except ValueError:
#     print("To nie jest liczba!")

print("""
W Data Science zamiast input() używamy:
- Argumentów linii poleceń (argparse, click)
- Plików konfiguracyjnych (YAML, JSON)
- Wczytywania danych z plików (CSV, Excel, SQL)
- Jupyter notebook widgets (dla interaktywnych analiz)
""")
```

> ⚠ **Warning**
>
> Funkcja `input()` ZAWSZE zwraca string! Jeśli potrzebujesz liczby, musisz użyć `int()` lub `float()` do konwersji. Niezrobienie tego to częsta przyczyna błędów u początkujących.

Schemat formatowania stringów



🌱 Aspekt środowiskowy

W tej lekcji zwracamy uwagę na:

1. Czytelny i prosty kod = mniejsze zużycie zasobów

Kod napisany zgodnie z best practices (dobre nazwy zmiennych, komentarze, PEP 8) jest:

- **Łatwiejszy do utrzymania** - mniej czasu na debugging i refactoring
- **Mniej podatny na błędy** - błędy powodują niepotrzebne ponowne uruchomienia
- **Bardziej efektywny** - doświadczeni programiści piszą bardziej optymalny kod

```
# ZŁY kod - niejasny, trudny do utrzymania, marnuje czas programisty
a = 150
b = 4
c = a / b  # Co to znaczy? Trzeba zgadywać!

# DOBRY kod - jasny, czytelny, oszczędza czas i energię
liczba_probek = 150
liczba_cech = 4
srednia_cech_na_probke = liczba_probek / liczba_cech

# Jasny kod = szybsze zrozumienie = mniej czasu na debugging = mniej zużytej
energii!
```

2. Efektywne operacje na stringach

```
# NIEEFEKTYWNE - tworzenie wielu tymczasowych obiektów
raport = ""
for i in range(1000):
    raport = raport + f"Wiersz {i}\n"  # Za każdym razem tworzy NOWY string!
# To zużywa dużo pamięci i czasu procesora

# EFEKTYWNE - użycie listy i join()
wiersze = []
for i in range(1000):
    wiersze.append(f"Wiersz {i}\n")
raport = "".join(wiersze)  # Jeden raz tworzy wynikowy string
# To jest ZNACZNIE szybsze i zużywa mniej pamięci
```

3. Unikanie niepotrzebnych konwersji typów

```
# NIEEFEKTYWNE - wielokrotna konwersja
liczba = 42
tekst = str(liczba)  # int -> str
znowu_liczba = int(tekst)  # str -> int
znowu_tekst = str(znowu_liczba)  # int -> str
# Każda konwersja kosztuje czas procesora i pamięć

# EFEKTYWNE - przemyślana struktura danych
liczba = 42
```

```
# Używaj liczby dopóki potrzebujesz obliczeń
wynik = liczba * 2
# Konwertuj na string TYLKO gdy naprawdę potrzebne
raport = f"Wynik: {wynik}"  # f-string robi konwersję automatycznie
```

4. Minimalizacja debugowania przez dobre praktyki

```
# Dobry kod redukuje liczbę błędów -> mniej ponownych uruchomień -> mniej
energii

# Używaj znaczących nazw zmiennych
# ZŁE:
x = 0.7  # Co to?
# DOBRE:
train_test_split_ratio = 0.7  # Jasne!

# Dodawaj komentarze do skomplikowanych operacji
# Formatowanie progu dla czytelności (2 miejsca po przecinku)
threshold = round(0.7891234, 2)

# Stosuj konwencje nazewnictwa (snake_case dla zmiennych)
liczba_epoch = 100  # DOBRE
liczbaEpoch = 100   # ZŁE (camelCase w Pythonie)
LiczbaEpoch = 100   # ZŁE (PascalCase - tylko dla klas)
```

> ✓ **Success**
>
> **Green IT Principle**: Kod, który jest łatwy do zrozumienia i utrzymania, pośrednio zmniejsza zużycie energii, bo redukuje czas debugowania, liczbe ponownych uruchomień i frustrację programistów (która prowadzi do dłuższej pracy przy komputerze).

📊 Praca z Prawdziwymi Danymi

Dataset: Podstawowe statystyki Iris (bez bibliotek)

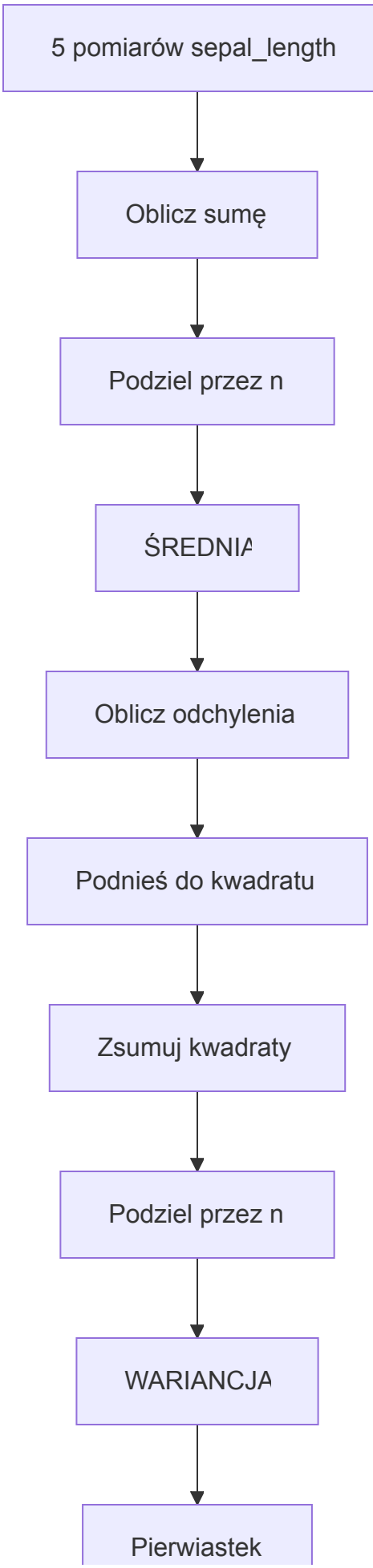
Pokażemy jak używać poznanych typów i operatorów do analizy danych na przykładzie kilku pomiarów z datasetu Iris.

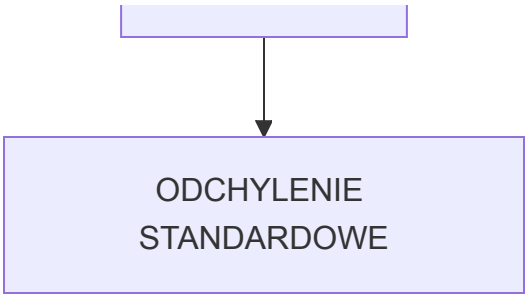
```
# Dataset Iris - próbka 5 kwiatów gatunku Setosa
# Kolumny: sepal_length (długość działki kielicha w cm)
```

```python
# Dane - ręcznie wprowadzone wartości
sepal_1 = 5.1
sepal_2 = 4.9
sepal_3 = 4.7
sepal_4 = 4.6
sepal_5 = 5.0

print("="*60)
print("ANALIZA DATASETU IRIS - Iris Setosa")
print("Cecha: Sepal Length (cm)")
print("="*60)

# Liczba pomiarów
n = 5
print(f"\nLiczba próbek: {n}")

# Wyświetlenie wszystkich pomiarów
print(f"\nPomiary:")
print(f"  Próbka 1: {sepal_1} cm")
print(f"  Próbka 2: {sepal_2} cm")
print(f"  Próbka 3: {sepal_3} cm")
print(f"  Próbka 4: {sepal_4} cm")
print(f"  Próbka 5: {sepal_5} cm")

# Obliczanie sumy
suma = sepal_1 + sepal_2 + sepal_3 + sepal_4 + sepal_5
print(f"\nSuma: {suma} cm")

# Średnia arytmetyczna
srednia = suma / n
print(f"Średnia (mean): {srednia:.2f} cm")

# Minimum i maximum
minimum = min(sepal_1, sepal_2, sepal_3, sepal_4, sepal_5)
maximum = max(sepal_1, sepal_2, sepal_3, sepal_4, sepal_5)
print(f"Minimum: {minimum} cm")
print(f"Maximum: {maximum} cm")
print(f"Zakres (range): {maximum - minimum:.2f} cm")

# Odchylenia od średniej
odch_1 = sepal_1 - srednia
odch_2 = sepal_2 - srednia
odch_3 = sepal_3 - srednia
odch_4 = sepal_4 - srednia
odch_5 = sepal_5 - srednia
```

```python
print(f"\nOdchylenia od średniej:")
print(f"  Próbka 1: {odch_1:+.2f} cm")  # + pokazuje znak
print(f"  Próbka 2: {odch_2:+.2f} cm")
print(f"  Próbka 3: {odch_3:+.2f} cm")
print(f"  Próbka 4: {odch_4:+.2f} cm")
print(f"  Próbka 5: {odch_5:+.2f} cm")

# Suma odchyleń (powinna być ≈0)
suma_odchylen = odch_1 + odch_2 + odch_3 + odch_4 + odch_5
print(f"\nSuma odchyleń: {suma_odchylen:.10f}")  # Bardzo blisko zera!

# Wariancja - średnia kwadratów odchyleń
kwadrat_odch_1 = odch_1 ** 2
kwadrat_odch_2 = odch_2 ** 2
kwadrat_odch_3 = odch_3 ** 2
kwadrat_odch_4 = odch_4 ** 2
kwadrat_odch_5 = odch_5 ** 2

suma_kwadratow = kwadrat_odch_1 + kwadrat_odch_2 + kwadrat_odch_3 +
kwadrat_odch_4 + kwadrat_odch_5
wariancja = suma_kwadratow / n
print(f"\nWariancja: {wariancja:.4f}")

# Odchylenie standardowe - pierwiastek z wariancji
odchylenie_std = wariancja ** 0.5  # ** 0.5 to pierwiastek kwadratowy
print(f"Odchylenie standardowe: {odchylenie_std:.4f} cm")

# Interpretacja
print(f"\n{'='*60}")
print("INTERPRETACJA:")
print(f"{'='*60}")
print(f"Średnia długość działki kielicha: {srednia:.2f} cm")
print(f"Typowe odchylenie od średniej: ±{odchylenie_std:.2f} cm")
print(f"Zakres wartości: [{minimum}, {maximum}] cm")
print(f"{'='*60}")
```

Wizualizacja prostych obliczeń


5 pomiarów sepal_length
Oblicz sumę
Podziel przez n
ŚREDNIA
Oblicz odchylenia
Podnieś do kwadratu
Zsumuj kwadraty
Podziel przez n
WARIANCJA
Pierwiastek



> **Note**
>
> W praktyce Data Science używamy bibliotek NumPy i Pandas, które wykonują te obliczenia automatycznie. Ten przykład pokazuje matematykę "pod maską", którą później zrozumiesz głębiej.

🧪 Zadania do samodzielnej pracy

✏️ Zadania podstawowe (1-8)

1. ✏️ **Zadanie 1 – Kalkulator BMI**

   Napisz program, który obliczy wskaźnik BMI (Body Mass Index).

   Dane:

   - Waga: 70 kg
   - Wzrost: 1.75 m

   Wymagania:

   - Stwórz zmienne `waga` i `wzrost`
   - Oblicz BMI według wzoru: BMI = waga / (wzrost²)
   - Wyświetl wynik z 2 miejscami po przecinku
   - Dodaj interpretację: BMI < 18.5 (niedowaga), 18.5-24.9 (norma), ≥25 (nadwaga)

   Oczekiwany rezultat:

   ```
   BMI: 22.86
   Interpretacja: Norma
   ```

   (proste)

2. ✏️ **Zadanie 2 – Konwersja temperatur**

Napisz program konwertujący temperatury Celsius na Fahrenheit i Kelvin.

Dane:

- Temperatura: 23.5°C

Wymagania:

- Fahrenheit = Celsius × 9/5 + 32
- Kelvin = Celsius + 273.15
- Wyświetl wszystkie 3 wartości sformatowane z 1 miejscem po przecinku

Oczekiwany rezultat:

```
23.5°C = 74.3°F = 296.6 K
```

(proste)

3. ✏️ **Zadanie 3 – Podział zbioru danych**

Oblicz rozmiary zbiorów treningowego, walidacyjnego i testowego.

Dane:

- Całkowita liczba próbek: 1000
- Proporcje: 70% trening, 15% walidacja, 15% test

Wymagania:

- Użyj mnożenia i dzielenia całkowitego
- Upewnij się, że suma = 1000 (obsłuż resztę z dzielenia)
- Wyświetl rozmiary wszystkich zbiorów

(proste)

4. ✏️ **Zadanie 4 – Formatowanie nazw kolumn**

Ustandaryzuj nazwę kolumny z datasetu.

Dane:

- Nazwa raw: `" PATIENT_age_YEARS "`

Wymagania:

- Usuń białe znaki
- Zamień na małe litery

- Zastąp `_` przez `" "`
- Użyj title case

Oczekiwany rezultat:

```
Patient Age Years
```

(proste)

5. ✏️ **Zadanie 5 – Parsing ścieżki pliku**

   Wyodrębnij informacje ze ścieżki do pliku.

   Dane:

   - Ścieżka: `"data/processed/train_fold_03_normalized.csv"`

   Wymagania:

   - Wyodrębnij: folder, typ zbioru (train/test), numer fold, format pliku
   - Użyj metod `split()` i indeksowania
   - Wyświetl każdą informację osobno

   (proste)

6. ✏️ **Zadanie 6 – Dokładność w procentach**

   Sformatuj metryki modelu.

   Dane:

   - Accuracy: 0.9234
   - Precision: 0.8891
   - Recall: 0.9456

   Wymagania:

   - Wyświetl każdą metrykę w procentach z 2 miejscami po przecinku
   - Użyj f-strings z formatowaniem `:.2%`
   - Dodaj estetyczne wyrównanie kolumn

   (proste)

7. ✏️ **Zadanie 7 – Porównanie dwóch modeli**

   Określ, który model jest lepszy.

Dane:

- Model A: accuracy = 0.87, train time = 120s
- Model B: accuracy = 0.89, train time = 450s

Wymagania:

- Porównaj accuracy (różnica > 0.02 jest znacząca)
- Porównaj czas treningu
- Użyj operatorów logicznych do określenia "najlepszego kompromisu"
- Wyświetl rekomendację

(proste)

8. ✏️ **Zadanie 8 – Obliczanie learning rate**

Oblicz learning rate dla schedulera.

Dane:

- Początkowy learning rate: 0.1
- Współczynnik decay: 0.95
- Numer epoki: 10

Wymagania:

- Użyj wzoru: lr_new = lr_initial × (decay ^ epoch)
- Operator potęgowania `**`
- Wyświetl wynik w notacji naukowej (`.2e`)

Oczekiwany rezultat:

```
Learning rate po 10 epokach: 5.99e-02
```

(proste)

✏️ Zadania średnie (9-12)

9. ✏️ **Zadanie 9 – Normalizacja Min-Max**

Znormalizuj wartość do zakresu [0, 1].

Dane:

- Wartość: 75

- Minimum w kolumnie: 50
- Maximum w kolumnie: 100

Wymagania:

- Wzór: normalized = (value - min) / (max - min)
- Oblicz dla wartości 75, 50, 100, 62.5
- Sprawdź, czy wynik mieści się w [0, 1]
- Wyświetl tabelkę: wartość oryginalna | wartość znormalizowana

(średnie)

10. ✏️ **Zadanie 10 – Budowanie raportu modelu**

    Stwórz sformatowany raport wyników.

    Dane:

    - Nazwa modelu: "XGBoost Classifier"
    - Dataset: "Credit Card Fraud"
    - Liczba próbek: 284807
    - Accuracy: 0.9991
    - Precision: 0.8876
    - Recall: 0.7561
    - F1-Score: oblicz ze wzoru 2×(precision×recall)/(precision+recall)

    Wymagania:

    - Użyj f-strings do budowy raportu
    - Zastosuj wyrównanie kolumn
    - Liczba próbek z separatorem tysięcy
    - Metryki w procentach z odpowiednią precyzją
    - Raport w ramce z linii `=`

    (średnie)

11. ✏️ **Zadanie 11 – Sprawdzanie poprawności hiperparametrów**

    Zwaliduj hiperparametry przed treningiem.

    Dane:

    - learning_rate = 0.01
    - batch_size = 64
    - epochs = 100

- dropout = 0.5

Wymagania:

- Sprawdź czy learning_rate w zakresie (0, 1]
- Sprawdź czy batch_size jest potęgą 2 (użyj operacji bitowych lub logarytmów)
- Sprawdź czy epochs > 0 i < 1000
- Sprawdź czy dropout w zakresie [0, 1]
- Wyświetl "OK" lub komunikat błędu dla każdego parametru

(średnie)

12. ✏️ **Zadanie 12 – Estymacja czasu treningu**

Oblicz szacowany czas treningu modelu.

Dane:

- Liczba próbek: 50000
- Batch size: 128
- Liczba epok: 50
- Czas na jeden batch: 0.15 sekundy

Wymagania:

- Oblicz liczbę batchy na epokę (użyj dzielenia całkowitego i modulo)
- Oblicz całkowity czas w sekundach
- Skonwertuj na minuty i godziny
- Wyświetl: "Szacowany czas: X h Y min Z s"

(średnie)

🧠 Zadania wyzwanie (13-20)

13. 🧠 **Zadanie 13 – Detekcja anomalii prostą metodą**

Wykryj wartości odstające używając reguły 3-sigma.

Dane (5 pomiarów ciśnienia krwi):

- 120, 125, 118, 190, 122 mmHg

Wymagania:

- Oblicz średnią i odchylenie standardowe (ręcznie, bez bibliotek)

- Zidentyfikuj wartości, które odchylają się o więcej niż 3×std od średniej
- Oznacz je jako anomalie
- Wyświetl raport z wszystkimi wartościami i oznaczeniem anomalii

Oczekiwany rezultat:

```
Wartość: 120 – OK
Wartość: 125 – OK
Wartość: 118 – OK
Wartość: 190 – ANOMALIA!
Wartość: 122 – OK
```

(challenge)

14. 🧠 **Zadanie 14 – Generowanie nazw plików dla cross-validation**

Wygeneruj nazwy plików dla k-fold cross-validation.

Dane:

- Dataset: "iris"
- Liczba folds: 5
- Typy: train, test

Wymagania:

- Użyj pętli for (zakres od 1 do k)
- Generuj nazwy w formacie: `iris_fold_01_train.csv`
- Numeracja z zerem wiodącym (01, 02, ..., 05)
- Wyświetl wszystkie nazwy (10 plików total)

Wskazówka: Użyj f-stringa z formatowaniem `:02d`

(challenge)

15. 🧠 **Zadanie 15 – Standaryzacja Z-score**

Standaryzuj wartości używając wzoru z-score.

Dane (wagi pacjentów w kg):

- 65.5, 78.2, 54.8, 92.1, 70.0

Wymagania:

- Oblicz średnią i odchylenie standardowe

- Dla każdej wartości oblicz z-score: z = (x - mean) / std
- Sprawdź, że średnia z-scores ≈ 0 i odchylenie ≈ 1
- Wyświetl oryginalną wartość i jej z-score
- Zinterpretuj: z > 2 (wartość bardzo wysoka), |z| < 1 (typowa), z < -2 (bardzo niska)

(challenge)

16. 🧠 **Zadanie 16 – Budowanie confusion matrix string**

    Stwórz tekstową reprezentację confusion matrix.

    Dane (klasyfikacja binarna):

    - True Positives: 45
    - True Negatives: 38
    - False Positives: 7
    - False Negatives: 10

    Wymagania:

    - Oblicz accuracy, precision, recall, F1-score
    - Narysuj confusion matrix używając stringów i formatowania
    - Użyj wyrównania do stworzenia czytelnej tabeli
    - Wyświetl metryki pod tabelą

    Przykład formatu:

```
              Predicted
              0      1
Actual  0   [38]   [7]
        1   [10]   [45]
```

(challenge)

17. 🧠 **Zadanie 17 – Parsing log-a treningowego**

    Wyodrębnij informacje z loga treningu modelu.

    Dane:

    - Log: `"Epoch 23/100 - loss: 0.3421 - accuracy: 0.8934 - val_loss: 0.4102 - val_accuracy: 0.8712"`

    Wymagania:

    - Użyj metod split(), strip(), indeksowania

- Wyodrębnij: numer epoki, total epok, loss, accuracy, val_loss, val_accuracy
- Skonwertuj wartości string na odpowiednie typy (int/float)
- Oblicz różnicę między train accuracy a val accuracy (oznaka overfittingu)
- Wyświetl sformatowany raport

(challenge)

18. 🧠 **Zadanie 18 – Walidacja formatu danych wejściowych**

    Sprawdź poprawność danych wejściowych przed analizą.

    Dane:

    - `" 45.7 "` - wiek pacjenta
    - `"male"` - płeć
    - `"120/80"` - ciśnienie krwi
    - `" 175.5cm "` - wzrost

    Wymagania:

    - Wyczyść białe znaki
    - Sprawdź czy wiek to liczba float w zakresie (0, 120]
    - Sprawdź czy płeć to "male" lub "female"
    - Rozdziel ciśnienie na skurczowe/rozkurczowe i zwaliduj zakresy
    - Wyodrębnij wartość liczbową ze wzrostu (usuń "cm")
    - Wyświetl raport walidacji dla każdego pola

    (challenge)

19. 🧠 **Zadanie 19 – Kalkulator accuracy z threshold-em**

    Oblicz accuracy dla różnych progów klasyfikacji.

    Dane (prawdopodobieństwa i prawdziwe etykiety):

    - Prawdopodobieństwa: 0.23, 0.67, 0.89, 0.34, 0.91, 0.12, 0.78, 0.45
    - Prawdziwe klasy: 0, 1, 1, 0, 1, 0, 1, 0

    Wymagania:

    - Dla threshold = 0.5: jeśli prob >= 0.5 → klasa 1, inaczej klasa 0
    - Porównaj predykcje z prawdziwymi klasami
    - Oblicz accuracy = (liczba poprawnych) / (liczba wszystkich)
    - Powtórz dla threshold = 0.3, 0.5, 0.7
    - Wyświetl accuracy dla każdego threshold i wskaż optymalny

(challenge)

20. 🧠 **Zadanie 20 – Generowanie SQL query z Python**

    Wygeneruj zapytanie SQL używając zmiennych Python.

    Dane:

    - Tabela: "patients"
    - Kolumny do wybrania: ["patient_id", "age", "diagnosis"]
    - Warunek 1: age > 40
    - Warunek 2: diagnosis w ['diabetes', 'hypertension']
    - Sortowanie: według age malejąco
    - Limit: 100 wierszy

    Wymagania:

    - Użyj f-strings do budowy zapytania SELECT
    - Poprawne formatowanie SQL (wielkie litery dla słów kluczowych)
    - Lista diagnoz jako string: "('diabetes', 'hypertension')"
    - Użyj join() do łączenia kolumn
    - Wyświetl gotowe zapytanie, które można skopiować do SQL

    Oczekiwany format:

```sql
SELECT patient_id, age, diagnosis
FROM patients
WHERE age > 40 AND diagnosis IN ('diabetes', 'hypertension')
ORDER BY age DESC
LIMIT 100;
```

    (challenge)

📚 Podsumowanie

W tej lekcji nauczyliśmy się podstaw programowania w Pythonie:

Kluczowe koncepcje:

- **Zmienne i typy danych** - int, float, string, bool
- **Operatory** - arytmetyczne (+, -, *, /, //, %, **), porównania (>, <, ==, !=), logiczne (and, or, not)

- **Formatowanie stringów** - f-strings jako najlepsze narzędzie
- **Konwersje typów** - int(), float(), str()
- **Metody stringów** - split(), join(), strip(), replace(), upper(), lower()

Powiązanie z poprzednimi lekcjami:

- **Lekcja 1**: Poznaliśmy typy zadań ML - teraz umiemy zapisać ich parametry jako zmienne
- **Lekcja 2**: Konfigrurowaliśmy środowisko - teraz piszemy w nim pierwszy kod
- **Lekcja 3**: Git pozwala wersjonować kod - teraz mamy co wersjonować!

Przygotowanie do następnych lekcji:

- Następnie poznamy **kolekcje** (listy, słowniki) - struktury do przechowywania wielu wartości
- Później **pętle i logikę** - automatyzacja powtarzalnych operacji
- Potem **funkcje** - organizacja kodu w wielokrotnie używalne bloki
- Wszystko to przygotuje nas do pracy z **NumPy i Pandas** - bibliotek do analizy danych!

Refleksja nad aspektem środowiskowym:

Czysty, dobrze napisany kod to nie tylko kwestia estetyki - to odpowiedzialność środowiskowa:

- Kod łatwy do zrozumienia = mniej czasu na debugging = mniej zużytej energii
- Efektywne operacje (join zamiast konkatenacji w pętli) = mniejsze obciążenie CPU i RAM
- Dobre praktyki = mniej błędów = mniej ponownych uruchomień programów treningowych

Pamiętaj: Każda sekunda zaoszczędzonego czasu procesora to energia, która nie musiała zostać zużyta!

📖 Dodatkowe Zasoby

Dokumentacja oficjalna:

- Python Tutorial - Built-in Types - typy wbudowane
- PEP 8 - Style Guide for Python Code - konwencje pisania kodu
- Python String Format Cookbook - formatowanie stringów

Interaktywne platformy do ćwiczeń:

- Python Tutor - wizualizacja wykonywania kodu krok po kroku
- Exercism - Python Track - zadania z mentorami
- HackerRank - Python - challenges programistyczne

Polecane artykuły:

- Real Python - Basic Data Types - szczegółowy tutorial
- Real Python - F-Strings - wszystko o f-strings
- Python for Data Science - Variables - context DS

Narzędzia pomocnicze:

- Python Type Checker - sprawdzanie typów w kodzie
- Black - automatyczne formatowanie kodu
- Pylint - analiza jakości kodu

Dla ciekawskich - Green IT:

- Green Software Foundation - zasady zrównoważonego developmentu
- CodeCarbon - pomiar emisji CO_2 w ML (poznamy później!)